Projeto Integrador: Análise de Compatibilidade de Vagas

Projeto: global TalenT bridge

Professor Orientador: Aimar Martins Lopes

Integrantes:

Bianca Marcondes camargo	25028292
Eriane dos Santos Oliveira Dias	24026778
Erik dos Santos Raimundo	24026628
Gustavo Roberto Melo da Silva	24026770
Kauã Daniel de Aguiar Ferreira	24026846

Brasil, São Paulo 2026

Introdução: Para o projeto globalTalenTbridge, foi implementado um algoritmo de compatibilidade de currículo em relação a vagas de emprego.

Objetivo: O objetivo é agilizar e simplificar a busca por empregos, problema esse que vem se tornando cada vez mais presente em nossa sociedade.

Lógica do Algoritmo:

- O algoritmo recebe o currículo e utiliza NLP para normalizar seu currículo e a vaga cadastrada no nosso banco de dados.

```python
import fitz  # PyMuPDF
import docx
import os

def extract_text_from_pdf(caminho_pdf):
    """
    Extrai o texto de um arquivo PDF usando PyMuPDF.
    """
    try:
        documento = fitz.open(caminho_pdf)
        texto = ""
        for pagina in documento:
            texto += pagina.get_text()
        return texto
    except Exception as erro:
        print(f"Erro ao extrair PDF: {erro}")
        return ""

def extract_text_from_docx(caminho_docx):
    """
    Extrai o texto de um arquivo DOCX usando python-docx.
    """
    try:
        documento = docx.Document(caminho_docx)
        texto = "\n".join([paragrafo.text for paragrafo in documento.paragraphs])
        return texto
    except Exception as erro:
        print(f"Erro ao extrair DOCX: {erro}")
        return ""

def extract_text(caminho_arquivo):
    """
    Identifica o tipo do arquivo e extrai o texto.
    """
    _, extensao = os.path.splitext(caminho_arquivo)
    extensao = extensao.lower()

    if extensao == ".pdf":
        return extract_text_from_pdf(caminho_arquivo)
    elif extensao == ".docx":
        return extract_text_from_docx(caminho_arquivo)
    else:
        # trata outros formatos como texto puro ou não suportados
        try:
            with open(caminho_arquivo, "r", encoding="utf-8") as arquivo:
                return arquivo.read()
        except:
            return ""

if __name__ == "__main__":
    # Script de teste
    pass

# aposo parse, navegaremos ate ao arquivo nlp_processor.py para normalização
```

```python
import re
import unicodedata
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# ---------------------------------------------------------------------------
# Tech keywords: mapeamento de termos especiais + boost de peso
# ---------------------------------------------------------------------------
TECH_MAP = {
    'c#': 'csharp',
    'c++': 'cpp',
    '.net': 'dotnet',
    'node.js': 'nodejs',
    'react.js': 'reactjs',
    'vue.js': 'vuejs',
    'next.js': 'nextjs',
    'asp.net': 'aspnet',
}

TECH_KEYWORDS = {
    'css', 'html', 'javascript', 'typescript', 'csharp', 'cpp', 'dotnet',
    'sql', 'python', 'java', 'reactjs', 'reactnative', 'nodejs', 'vuejs',
    'nextjs', 'aspnet', 'flutter', 'dart', 'spark', 'airflow', 'terraform',
    'aws', 'azure', 'gcp', 'docker', 'kubernetes', 'kafka', 'rabbitmq',
    'oracle', 'mongodb', 'postgresql', 'redis', 'linux', 'git', 'api',
    'rest', 'graphql', 'fastapi', 'django', 'spring', 'angular', 'android',
    'ios', 'swift', 'kotlin', 'golang', 'rust', 'scala', 'hadoop',
    'machine', 'learning', 'backend', 'frontend', 'fullstack', 'devops',
    'cloud', 'microservices', 'agile', 'scrum',
}

# ---------------------------------------------------------------------------
# NLTK setup (idempotente)
# ---------------------------------------------------------------------------
def _ensure_nltk():
    resources = [
        ('corpora/stopwords', 'stopwords'),
        ('tokenizers/punkt_tab', 'punkt_tab'),
        ('tokenizers/punkt', 'punkt'),
    ]
    for path, pkg in resources:
        try:
            nltk.data.find(path)
        except LookupError:
            nltk.download(pkg, quiet=True)

_ensure_nltk()

STOP_WORDS = set(stopwords.words('english')) | set(stopwords.words('portuguese'))

# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------
def _normalize(text: str) -> str:
    """Remove acentos normalizando para NFD e descartando combining chars."""
    return unicodedata.normalize('NFD', text).encode('ascii', 'ignore').decode('ascii')


def clean_text(text: str) -> str:
    """
    Pipeline de limpeza de texto para PT-BR e EN:
    1. Normaliza encoding (remove acentos de forma limpa).
    2. Mapeia termos técnicos especiais (C#, C++, Node.js …).
    3. Tokeniza e filtra stopwords.
    4. Duplica termos técnicos para dar peso 2x no TF-IDF.
    """
    if not text:
        return ""

    # 1. Lowercase + normalização de acentos
    text = text.lower()
```

```python
    # 1. Lowercase + normalização de acentos
    text = text.lower()
    for original, replacement in TECH_MAP.items():
        text = text.replace(original, replacement)
    text = _normalize(text)

    # 2. Tokenização simples (mantém apenas alfanuméricos)
    tokens = re.findall(r'[a-z0-9]+', text)

    # 3. Remove stopwords e tokens muito curtos (<=1 char)
    tokens = [t for t in tokens if len(t) > 1 and t not in STOP_WORDS]

    # 4. Boost: duplica techs + tokens longos (>=5) que provavelmente são substantivos
    final_tokens = []
    for t in tokens:
        final_tokens.append(t)
        if t in TECH_KEYWORDS or len(t) >= 5:
            final_tokens.append(t)  # peso dobrado

    return " ".join(final_tokens)


if __name__ == "__main__":
    test_text = "Conhecimento intermediário em C#, C++, Node.js, CSS e Machine Learning."
    print(clean_text(test_text))

# apos a normalização, navegaremos ate ao arquivo model.py para a vetorização
```

- O processo de decisão funciona após a normalização, ele compara as palavras chave do currículo com as dependências da vaga.

```python
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer
from .nlp_processor import clean_text


def calculate_match_scores_bulk(resume_text: str, jobs: list[dict]) -> list[dict]:
    """
    Calcula a similaridade entre um currículo e MÚLTIPLAS vagas de uma vez,
    usando um único TF-IDF fitado com o corpus completo (currículo + todas as vagas).
    Isso garante um IDF significativo e scores muito mais precisos.

    Args:
        resume_text: Texto bruto do currículo.
        jobs: Lista de dicts com pelo menos as chaves 'title', 'description', 'requirements'.

    Returns:
        Lista de dicts com os campos originais da vaga + 'score' (0-100).
    """
    if not jobs or not resume_text:
        return []

    # 1. Limpa o texto do currículo
    clean_resume = clean_text(resume_text)

    # 2. Limpa cada vaga e monta o corpus completo
    job_texts = []
    for job in jobs:
        job_text = (
            f"{job.get('title', '')} {job.get('title', '')} "   # titulo 2x para boost
            f"{job.get('description', '')} "
            f"{job.get('requirements', '')}"
        )
        job_texts.append(clean_text(job_text))

    # 3. Corpus = [currículo, vaga_0, vaga_1, ..., vaga_n]
    corpus = [clean_resume] + job_texts

    # 4. Único vetorizador para o corpus inteiro
    vectorizer = TfidfVectorizer(
        ngram_range=(1, 2),   # unigramas + bigramas (captura "machine learning", "banco dados")
        min_df=1,
        sublinear_tf=True,    # log(TF+1) — reduz dominância de termos muito repetidos
    )
    try:
        tfidf_matrix = vectorizer.fit_transform(corpus)
    except ValueError:
        # Corpus vazio ou inválido
        return []

    # 5. Vetor do currículo (índice 0) vs. cada vaga (índices 1..n)
    resume_vector = tfidf_matrix[0:1]
    job_vectors = tfidf_matrix[1:]

    similarities = cosine_similarity(resume_vector, job_vectors)[0]

    # 6. Monta resultados enriquecidos
    results = []
    for i, job in enumerate(jobs):
        score = round(float(similarities[i]) * 100, 2)
        results.append({
            **job,
            "score": score,
        })

    return results
```

```python
def calculate_match_score(resume_text: str, job_description_text: str) -> float:
    """
    Compat: calcula score para um único par curriculo-vaga.
    Internamente usa a função bulk com corpus de 2 documentos.
    Preferir sempre calculate_match_scores_bulk para múltiplas vagas.
    """
    fake_job = {"title": "", "description": job_description_text, "requirements": ""}
    results = calculate_match_scores_bulk(resume_text, [fake_job])
    return results[0]["score"] if results else 0.0


if __name__ == "__main__":
    resume = "Experienced Software Engineer with proficiency in Python, FastAPI, and Machine Learning."
    jobs_sample = [
        {
            "title": "Python Developer",
            "description": "Looking for a Python developer with experience in AI and ML integrations.",
            "requirements": "Python, FastAPI, Machine Learning",
        },
        {
            "title": "Frontend Developer",
            "description": "React developer needed for UI work.",
            "requirements": "React, JavaScript, CSS",
        },
    ]
    for r in calculate_match_scores_bulk(resume, jobs_sample):
        print(f"{r['title']}: {r['score']}%")

# cálculo de similaridade completo, seguiremos ao arquivo main.py para a criação da API
```

- A estratégia prioriza encontrar palavras correspondentes com os requisitos solicitados pelo empregador, trazendo assim uma maior chance de seu currículo ser aprovado pelo empregador.

---

2. Resultados Aplicados (Entrada e Saída)

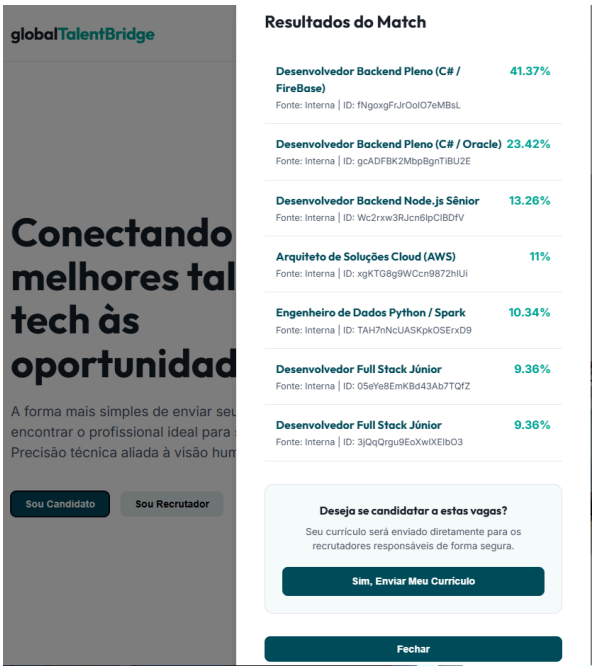
Entrada (Parâmetros definidos):

- Currículo DEV genérico
- Currículo DEV voltado a FireBase
- Currículo sem competências DEV

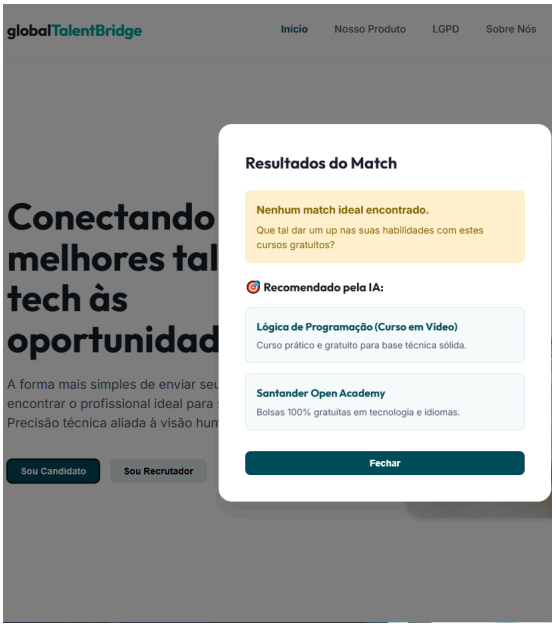
Exemplo de Saída (Resultados Gerados):

Currículo DEV genérico:

Currículo DEV FireBase:



Currículo sem competências DEV:



---

3. Tecnologias Utilizadas

• **Backend:** Python, FastAPI, PyMuPDF & python-docx, NLP: Spacy, NLTK, e Scikit-learn, Pandas

• **Frontend:** React 19, Vite, CSS Moderno, pdfjs-dist (para processamento de PDF no navegador).

• **Algoritmo de IA:** Processamento de Linguagem Natural (NLP) e Recuperação de Informação, utilizando mais especificamente:

TF-IDF (Term Frequency-Inverse Document Frequency), Similaridade de Cosseno, N-grams (Unigramas e Bigramas), Processamento de Texto (NLP Pipelines)

• **Base de Dados / Repositório:** [Firebase (Google):

Authentication, Firestore: Banco de dados NoSQL para armazenar vagas, currículos e recomendações, Admin SDK: Integração do backend Python com os serviços Firebase.

---

4. Arquitetura do Sistema

O sistema segue a seguinte arquitetura de funcionamento:

- Solicita a criação de um perfil para o envio do currículo.
- Após o login, o usuário envia o currículo no botão “candidatar-se".
- O algoritmo recebe o documento, identifica, normaliza e compara.
- É criado um modal para exibir a porcentagem de compatibilidade.

Arquivos principais do projeto:

- • main.py: é o ponto de entrada do servidor. Ele utiliza o framework FastAPI para gerenciar as rotas da aplicação.
- • matcher.py: É o "cérebro" da IA. Nele está programada a lógica que utiliza as bibliotecas scikit-learn.
- • firebase_config.py: Como o projeto utiliza uma arquitetura moderna na nuvem, os dados (vagas e currículos) não ficam em um arquivo local estático, mas sim no Firebase Firestore.
- • App.jsx: A interface visual é construída com React.